



Comparing Retrieval-Augmented LLM Pipelines for question Answer over a University Campus Digital Twin

Flavio Herrera

Abstract

- Goal: Answer natural-language questions over a university campus digital twin.
- Data: Events, building information, services, and user-generated reports such as wildlife sightings.
- Baseline: Sends condensed candidate items directly to a closed-source LLM for relevance ranking.
- Enhanced Pipeline: Adds lightweight retrieval using query expansion, searchable item encoding, lexical/fuzzy matching, field boosts, and nested-content relevance before LLM generation.

Problem

- Problem: Campus information is fragmented across many systems.
- Student Need: Students ask practical natural-language questions about locations, dining, events, and services.
- Search Limitation: Traditional search requires exact keywords and manual result stitching.
- Digital Twin Gap: A digital twin structures campus data, but students still need an interface that maps informal questions to accurate, grounded answers.

Research Question

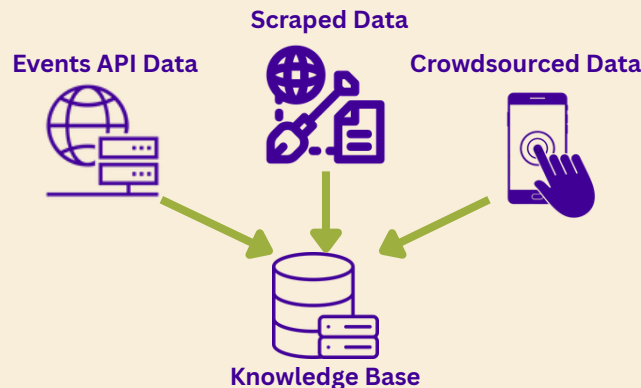
- Can retrieval and ranking reduce LLM reasoning load without sacrificing answer quality?
- RQ1: Does pre-LLM retrieval improve source ranking?
- RQ2: Does it improve answer relevance for campus queries?
- RQ3: What trade-offs emerge in cost, latency, and context size?

Poster and Code

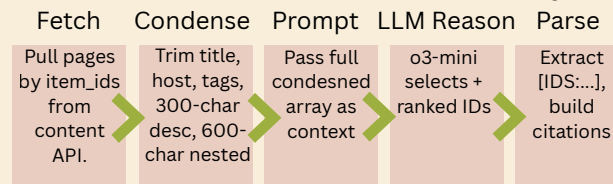
https://github.com/herreraflavio/uc_merced_campus_event_api_backend

Data & Knowledge Base

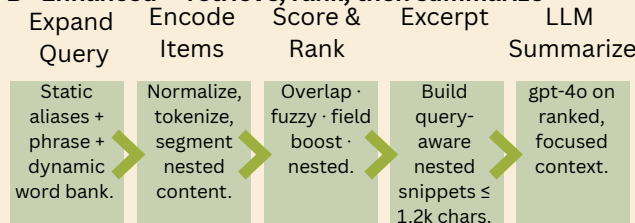
- Source: Live campus events API, manually scraped, crowdsourced
- Items: Events, dining, services, buildings, and other campus pages.
- Fields: Title, host, tags, description, location, and nested content.
- Nested Data: Menus, schedules, sections, and structured page details.
- Preprocessing: Normalize text, remove URLs, compact long fields, and encode records for retrieval.



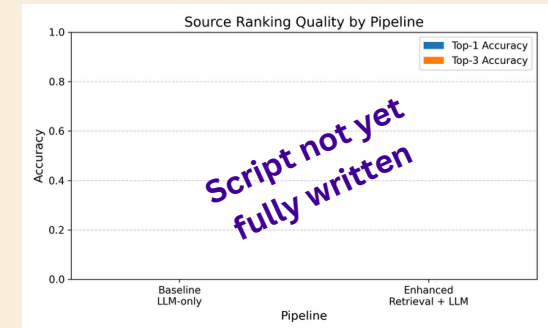
A) Baseline Condense → LLM Reasons over everything



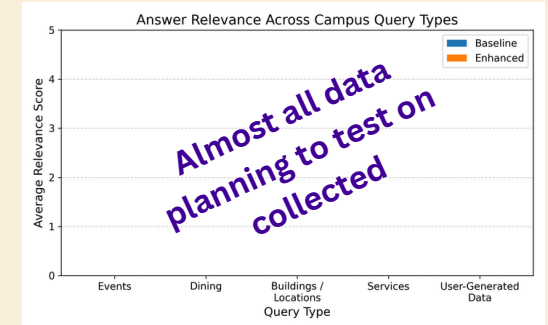
B • Enhanced • retrieve, rank, then summarize



1. Source Ranking Quality by Pipeline



2. Answer Relevance Across Query Types



3. Cost-Latency-Context Trade-off

